

Projektowanie aplikacji mobilnych

Ćwiczenia #5 – Networking

Grupa:

Ilias Abdeljalil (40028)

1. Wybrać publiczne API i dodać interfejs z potrzebnymi w aplikacji funkcjami (1pkt)

Na początku proszę znaleźć publiczne API, które oferuje możliwość pobierania przykładowych danych w formacie JSON oraz przynajmniej jeden endpoint typu POST (odbierający dane). API musi dać się dopasować jakoś do państwa aplikacji, ale nie musi pasować idealnie.

Zamiast zewnętrznego API, które nie pasuje do wybranej przeze mnie aplikacji, zrobiłem lokalne przechowywanie danych w formacie JSON.

Screeny z implementacją skryptu w kontenerze DI:

```
8 public class JsonHandler
9 {
10     private readonly string filePath = Path.Combine(path1: Application.persistentDataPath, path2: "Archive.json");
11
12     [1 usage]
13     public ArchiveModel ArchiveInit()
14     {
15         var archive = !File.Exists(filePath) ? new ArchiveModel() : LoadFromJson();
16
17         return archive;
18     }
19
20     [1 usage]
21     public void SaveToJson(ArchiveModel data)
22     {
23         try
24         {
25             var jsonString = JsonUtility.ToJson(data, prettyPrint: true);
26             File.WriteAllText(path: filePath, contents: jsonString);
27             Debug.Log(message: "Data saved successfully: " + filePath);
28         }
29         catch (Exception e)
30         {
31             Debug.LogError(message: "Failed to save data: " + e.Message);
32         }
33     }
34 }
```

```

19         public void SaveToJson(ArchiveModel data)
20         {
21             try
22             {
23                 var json:string = JsonUtility.ToJson(data, prettyPrint: true);
24                 File.WriteAllText(path: filePath, contents: json);
25                 Debug.Log(message: "Data saved successfully: " + filePath);
26             }
27             catch (Exception e)
28             {
29                 Debug.LogError(message: "Failed to save data: " + e.Message);
30             }
31         }
32     }

```

```

33     private ArchiveModel LoadFromJson()
34     {
35         try
36         {
37             if (File.Exists(filePath))
38             {
39                 var json:string = File.ReadAllText(filePath);
40                 var data = JsonUtility.FromJson<ArchiveModel>(json);
41                 Debug.Log(message: "Data loaded from JSON");
42                 return data;
43             }
44
45             Debug.LogWarning(message: "File is not found: " + filePath);
46             return null;
47         }
48         catch (Exception e)
49         {
50             Debug.LogError(message: "Something went wrong: " + e.Message);
51             return null;
52         }
53     }

```

2. Zaimplementować Networking w aplikacji – tylko najprostszy endpoint (3pkt)

Zaprojektowany w poprzednim zadaniu interfejs/protokół należy zaimplementować. Polecam wykorzystać w tym celu biblioteki typu Ktor Client (Android) czy Alamofire (iOS). W ramach tego zadania wystarczy zaimplementować tylko jeden, najprostszy endpoint (jedną funkcję interfejsu). Najlepiej, jeśli będzie to endpoint typu GET.

Mając zaimplementowaną klasę, należy dodać ją do frameworku Dependency Injection, który został dodany w ramach poprzednich zajęć.

Ostatni krok to wstrzyknięcie tej klasy do ViewModel'u ekranu, w którym przyda się to zapytanie i wykorzystanie go do pokazania pobranych danych na ekranie.

Zmieniony model danych oraz użycie JsonHandler:

```

6      [Serializable]
      10 usages  1 dovara41 *  1 exposing API
7      public class ArchiveModel
8      {
9          public List<TextDocument> textDocuments = new();  * Serializable
10         public List<ListDocument> listDocuments = new();  * Serializable
11     }
12
13     [Serializable]
      3 usages  1 dovara41 +1 *
14     ^o public class TextDocument : Document
15     {
16         public string text;  * Serializable
17     }
18
19     [Serializable]
      3 usages  1 dovara41 +1 *
20     ^o public class ListDocument : Document
21     {
22         public List<string> elements = new();  * Serializable
23     }
24
25     [Serializable]
      4 usages  2 inheritors  1 dovara41 +1 *
26     ^o public class Document
27     {
28         public string title;  * Serializable
29     }

```

```

14
15     private JsonHandler jsonHandler;
16     private ArchiveModel archiveModel;
17     private ArchiveViewModel archiveViewModel;
18
19     * Event function  1 alias *
20     private void Awake()
21     {
22         jsonHandler = new JsonHandler();
23         archiveModel = jsonHandler.ArchiveInit();
24
25         archiveViewModel = new ArchiveViewModel(archiveModel, archiveView, listItem);
26
27         archiveView.Construct(archiveModel, archiveViewModel);
28
29         archiveViewModel.UpdateFoldensCount();
30     }
31
32     * Event function  1 new *
33     private void OnDestroy()
34     {
35         jsonHandler.SaveToJson(archiveModel);
36     }

```

3. Zaimplementować pozostałe zapytania potrzebne w aplikacji (2pkt)

Należy zaimplementować pozostałe funkcje interfejsu – przynajmniej 2. Jedna z nich musi być zapytanie typu POST (wysyłanie danych na serwer).

Zaimplementowane funkcje, należy również wykorzystać w państwa aplikacji.

Screeny potwierdzające poprawną implementację: